USART
(Universal Synchronous/Asynchronous Receiver and Transmitter)
Register Description

Embedded Controller

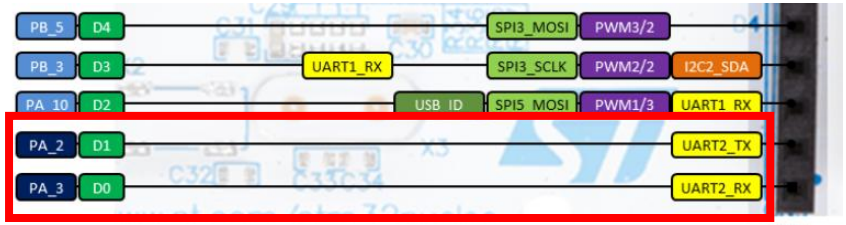
Prof. Young-Keun Kim

USART Register

- **USARTx Configuration**

MCU: STM32F411 RE

- Has 3x USART: USART2 // USART1, USART6
- Has 5x SPI: SPI2,SPI3 // SPI5, SPI4,SPI1



TX: PA2 RX: PA3

<USART2: COMx port via ST-Link(USB)>

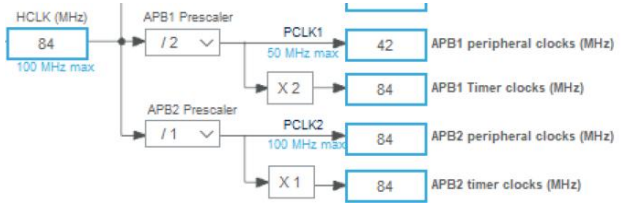
RCC_APB1ENR: Clock Enable

- RCC_APB1[17]: USART2EN
- RCC_APB1[15]: SPI3EN
- RCC_APB1[14]: SPI2EN

APB1_CLK is max 50MHz
APB2_CLK is max 100MHz

RCC_APB2ENR: Clock Enable

- RCC_APB2[4]: USART1EN
- RCC_APB2[5]: USART6EN
- RCC_APB2[12]: SPI1EN
- RCC_APB2[13]: SPI4EN
- RCC_APB2[20]: SPI5EN

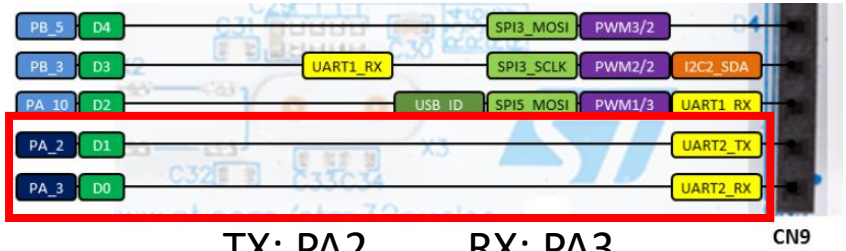


USART Register

USARTx Configuration

GPIO Pin setting

1. Alternative Function mode selection for Pin_y in GPIOx
2. Alternative Function Register for USARTx
3. GPIO pin configuration as No pull up, No pull down



TX: PA2 RX: PA3

<USART2: COMx port via ST-Link(USB)>

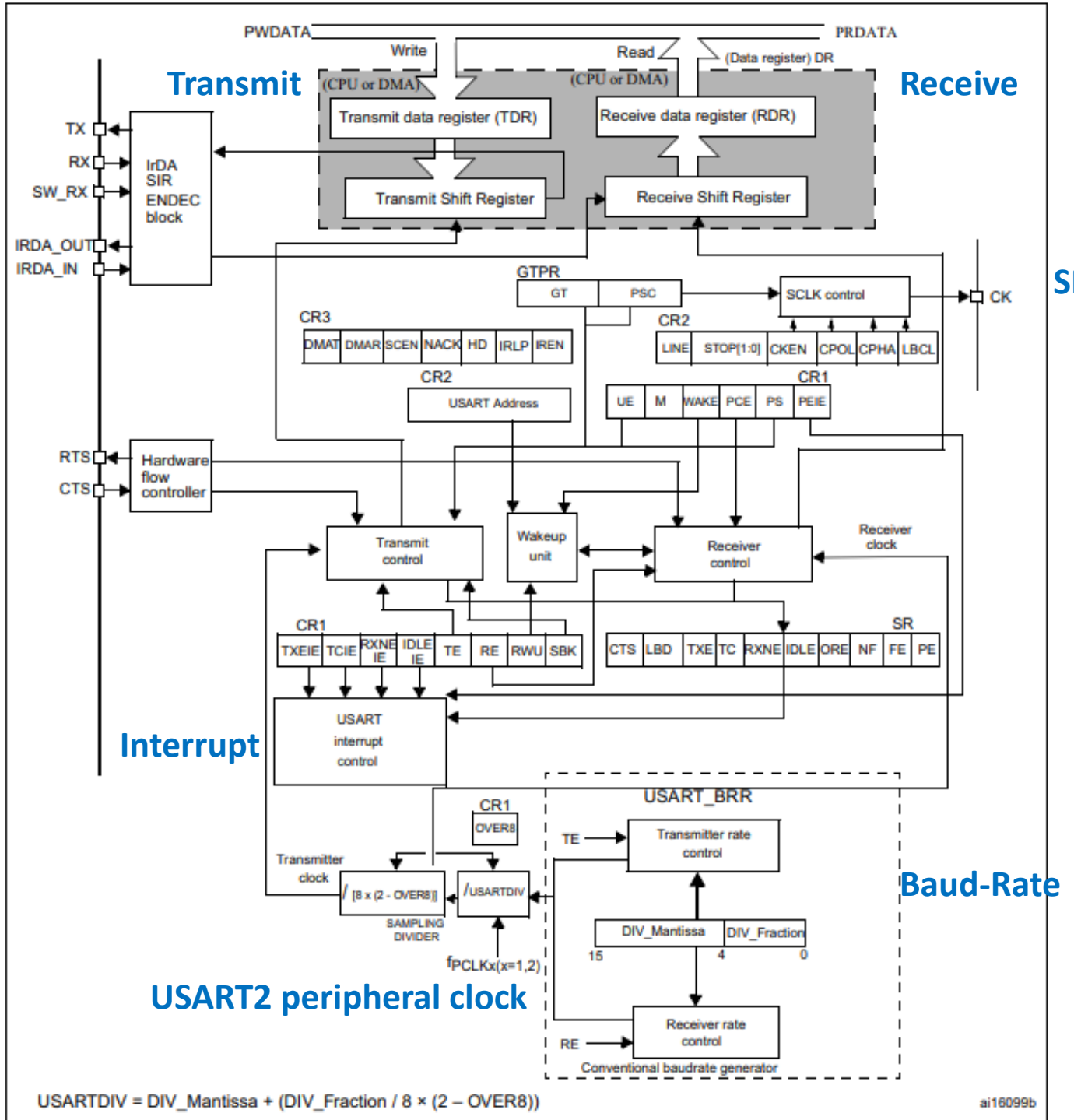
USART setting (x=2, 1,6)

1. Enable USARTx peripheral clock — **(RCC → APBxENR : USARTxEN) //** y=1 if x=2
2. Disable USARTx — **(USARTx → CR1 : UE)**
3. Configure Parity mode , Word Length, Oversampling mode — **(USARTx → CR1 : PCE // M // OVER8)**
4. Set the Stop Bit — **(USARTx → CR2 : STOP)**
5. Configure Baud-Rate(USARTDIV) — **(USARTx → BRR)**
6. Enable Transmitter, Receiver, and USARTx — **(USARTx → CR1 : TE, RE, UE)**

Transmit and receive USART data

1. Check the status of USART — **(USARTx → SR : TXE // TC)**, **(USARTx → SR : RXNE)**
2. Transmit or receive the data — **(USARTx → DR)**

- **USARTx Registers**



USART Register

- **USARTx Configuration**

1. RCC: Enable USART2 peripheral clock **(RCC → APB1ENR : USART2EN)**

Example: USART2

6.3.11 RCC APB1 peripheral clock enable register (RCC_APB1ENR)

Address offset: 0x40

Reset value: 0x0000 0000

Access: no wait state, word, half-word and byte access.

```
// Enable the clock of USART2
RCC->APB1ENR |= RCC_APB1ENR_USART2EN;
```

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved			PWR EN	Reserved				I2C3 EN	I2C2 EN	I2C1 EN	Reserved			USART2 EN	Reser-ved
			rw					rw	rw	rw				rw	

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SPI3 EN	SPI2 EN	Reserved		WWDG EN	Reserved							TIM5 EN	TIM4 EN	TIM3 EN	TIM2 EN
rw	rw			rw								rw	rw	rw	rw

USART Initialization

- **USARTx Configuration**

2. Disable USART2 **(USART2 → CR1 : UE)**

- Need to disable before configuration setting

Example: USART2

```
USARTx->CR1 &= ~USART_CR1_UE;
```

19.6.4 Control register 1 (USART_CR1)

Address offset: 0x0C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OVER8	Reserved	UE	M	WAKE	PCE	PS	PEIE	TXEIE	TCIE	RXNEIE	IDLEIE	TE	RE	RWU	SBK
rw	Res.	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 13 **UE**: USART enable

When this bit is cleared, the USART prescalers and outputs are stopped and the end of the current byte transfer in order to reduce power consumption. This bit is set and cleared by software.

0: USART prescaler and outputs disabled

1: USART enabled

USART Initialization

- **USARTx Configuration**

3. Configure Parity mode , Word Length, Oversampling **(USART2 → CR1 : PCE // M // OVER8)**

19.6.4 Control register 1 (USART_CR1)

Address offset: 0x0C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OVER8	Reserved	UE	M	WAKE	PCE	PS	PEIE	TXEIE	TCIE	RXNEIE	IDLEIE	TE	RE	RWU	SBK
rw	Res.	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 10 **PCE**: Parity control enable

This bit selects the hardware parity control (generation and detection). When the parity control is enabled, the computed parity is inserted at the MSB position (9th bit if M=1; 8th bit if M=0) and parity is checked on the received data. This bit is set and cleared by software. Once it is set, PCE is active after the current byte (in reception and in transmission).

0: Parity control disabled

1: Parity control enabled

Bit 12 **M**: Word length

This bit determines the word length. It is set or cleared by software.

0: 1 Start bit, 8 Data bits, n Stop bit

1: 1 Start bit, 9 Data bits, n Stop bit

Note: The M bit must not be modified during a data transfer (both transmission and reception)

Bit 15 **OVER8**: Oversampling mode

0: oversampling by 16

1: oversampling by 8

Note: Oversampling by 8 is not available in the Smartcard, IrDA and LIN modes: when SCEN=1,IREN=1 or LINEN=1 then OVER8 is forced to '0 by hardware.

Example: USART2

- No parity, 8-bit word
- Oversample by 16 (default)

```
USARTx->CR1 &= ~USART_CR1_M;
USARTx->CR1 &= ~USART_CR1_PCE;
USARTx->CR1 &= ~USART_CR1_OVER8;
```

USART Initialization

- **USARTx Configuration**

4. Set the Stop bit **(USART2 → CR2 : STOP)**

19.6.5 Control register 2 (USART_CR2)

Address offset: 0x10

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	LINEN	STOP[1:0]		CLKEN	CPOL	CPHA	LBCL	Res.	LBDIE	LBDL	Res.	ADD[3:0]			
	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw	rw	rw

Bits 13:12 **STOP**: STOP bits

These bits are used for programming the stop bits.

00: 1 Stop bit

01: 0.5 Stop bit

10: 2 Stop bits

Note: 11: 1.5 Stop bit

```
USARTx->CR2 &= ~USART_CR2_STOP;
```

Example: USART2

- No parity, 8-bit word(default)
- Oversample by 16 (default)
- 1 Stop Bit (default)

USART Initialization

● USARTx Configuration

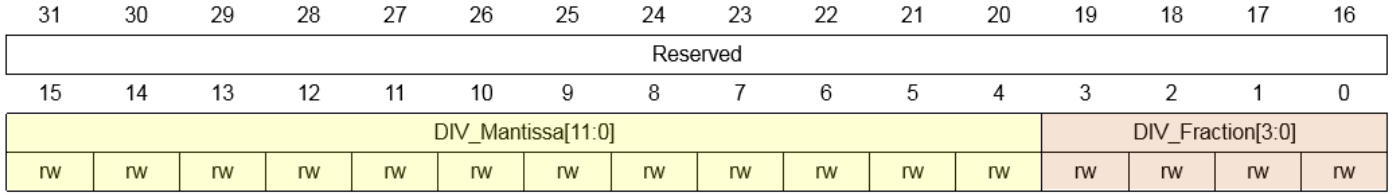
5. Configure USARTDIV depending on the baud-rate. (**USARTx → BRR)**

19.6.3 Baud rate register (USART_BRR)

Note: The baud counters stop counting if the TE or RE bits are disabled respectively.

Address offset: 0x08

Reset value: 0x0000 0000



USARTDIV 정수 부분

USARTDIV 소수점 아래

Example: USART2

- No parity, 8-bit word(default)
- Oversample by 16 (default)
- 1 Stop Bit (default)
- 9600 Baud-Rate

```
#define BAUD_9600      0
#define BAUD_38400     1
#define BAUD_57600     2
#define BAUD_115200    3
```

$$\text{USARTDIV} = \frac{f_{\text{APBx}}}{16 \times \text{Baud Rate}}$$

$$\text{USARTDIV} = \frac{42{,}000{,}000}{16 \times 9600} = 273.44 \rightarrow$$ Mantissa = d273 (0x111), Fraction = d0.44*16 (0x7)

USART Initialization

● USARTx Configuration

5. Configure USARTDIV depending on the baud-rate. (**USARTx → BRR)**

If PLL=84MHz
USART2: f_{APB} = APB1 Clock = 42MHz
USART1,6: f_{APB} = APB2 Clock = 84MHz

Example: Baud Rate = 9600 [bps], for USART2

(Method 1)

$$\text{USARTDIV} = \frac{42{,}000{,}000}{16 \times 9600} = 273.44$$

Mantissa = 273 = 0x111

Fraction = 0.4375 x 16 = 0x7

USART_BRR = (Mantissa <<4) | Fraction = 0x1117

(Method 2)

$$(\text{USARTDIV} \ll 4) = \frac{42{,}000{,}000}{9600} = 4375$$

BRR=4375 = 0x1117

USART_BRR =BRR

Note:

0x1117 = (0x111 << 4)+ 0x7 = 4375

USART Initialization

- **USARTx Configuration**

6. Enable Transmitter, Receiver, and USARTx **(USARTx → CR1 : TE, RE, UE)**

Example: USART2
- No parity, 8-bit word(default)
- Oversample by 16 (default)
- 1 Stop Bit (default)
- 9600 Baud-Rate

19.6.4 Control register 1 (USART_CR1)

Address offset: 0x0C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OVER8	Reserved	UE	M	WAKE	PCE	PS	PEIE	TXEIE	TCIE	RXNEIE	IDLEIE	TE	RE	RWU	SBK
rw	Res.	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 3 **TE**: Transmitter enable

This bit enables the transmitter. It is set and cleared by software.

0: Transmitter is disabled

1: Transmitter is enabled

Note: During transmission, a "0" pulse on the TE bit ("0" followed by "1") sends a preamble (idle line) after the current word, except in smartcard mode.

When TE is set, there is a 1 bit-time delay before the transmission starts.

Bit 2 **RE**: Receiver enable

This bit enables the receiver. It is set and cleared by software.

0: Receiver is disabled

1: Receiver is enabled and begins searching for a start bit

Bit 13 **UE**: USART enable

When this bit is cleared, the USART prescalers and outputs are stopped and the end of the current byte transfer in order to reduce power consumption. This bit is set and cleared by software.

0: USART prescaler and outputs disabled

1: USART enabled

```
USARTx->CR1  |= (USART_CR1_RE | USART_CR1_TE);
USARTx->CR1  |= USART_CR1_UE;
```

Transmit and receive data

- **Reading / Transmitting Data**

  - Polling method
    - Simple, but not efficient
  - Interrupt method
    - More efficient, but not for high data transfer

  - *DMA method
    - Most complex, but most efficient for high data transfer

Transmit and receive data

Reading / Transmitting Data

1. (for **TX**): Check the status of USART **(USARTx → SR : TXE , TC)**

- TXE==1 : TRD is transferred to Shift register
- TCE==1 : Transmission is completed

19.6.1 Status register (USART_SR)

Address offset: 0x00

Reset value: 0x00C0 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved						CTS	LBD	TXE	TC	RXNE	IDLE	ORE	NF	FE	PE
						rc_w0	rc_w0	r	rc_w0	rc_w0	r	r	r	r	r

Bit 7 **TXE**: Transmit data register empty

This bit is set by hardware when the content of the TDR register has been transferred into the shift register. An interrupt is generated if the TXEIE bit =1 in the USART_CR1 register. It is cleared by a write to the USART_DR register.

0: Data is not transferred to the shift register

1: Data is transferred to the shift register)

TD is NOT empty

TD is empty, ready for next TX

Note: This bit is used during single buffer transmission.

Bit 6 **TC**: Transmission complete

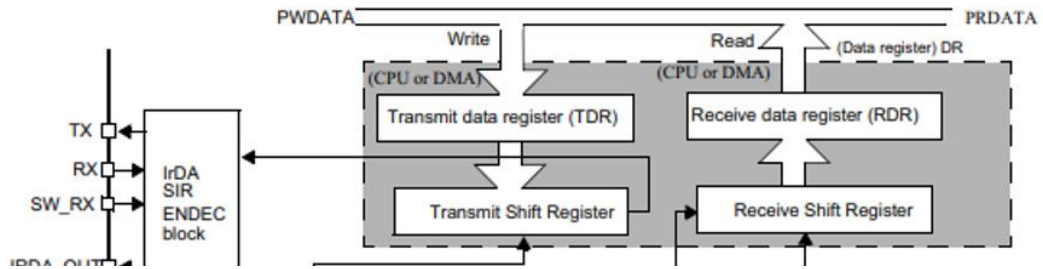
This bit is set by hardware if the transmission of a frame containing data is complete and if TXE is set. An interrupt is generated if TCIE=1 in the USART_CR1 register. It is cleared by a software sequence (a read from the USART_SR register followed by a write to the USART_DR register). The TC bit can also be cleared by writing a '0' to it. This clearing sequence is recommended only for multibuffer communication.

0: Transmission is not complete

1: Transmission is complete

TC is still on going

TC is done. Ready for next TX




```
void USART_write(USART_TypeDef * USARTx, uint8_t *buffer, uint32_t nBytes) {
    int i;

    // TXE is set by hardware when the content of the TDR
    // register has been transferred into the shift register.

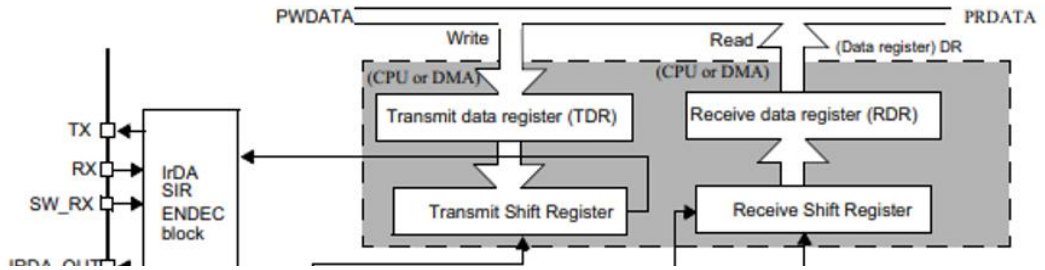
    for (i = 0; i < nBytes; i++) {
        // wait until TXE (TX empty) bit is set
        while (!(USARTx->SR & USART_SR_TXE));
        // Writing USART_DR automatically clears the TXE flag
        USARTx->DR = buffer[i] & 0xFF;
        USART_delay(300);
    }
    // wait until TC bit is set
    while (!(USARTx->SR & USART_SR_TC));
    // TC bit clear
    USARTx->SR &= ~USART_SR_TC;
}
```

Transmit and receive data

● Reading / Transmitting Data

2. (for **RX**): Check the status of USART **(USARTx → SR : RXNE)**

- RXNE==1 : Receive Data is ready to read



19.6.1 Status register (USART_SR)

Address offset: 0x00

Reset value: 0x00C0 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved						CTS	LBD	TXE	TC	RXNE	IDLE	ORE	NF	FE	PE
						rc_w0	rc_w0	r	rc_w0	rc_w0	r	r	r	r	r

Bit 5 **RXNE**: Read data register not empty

This bit is set by hardware when the content of the RDR shift register has been transferred to the USART_DR register. An interrupt is generated if RXNEIE=1 in the USART_CR1 register. It is cleared by a read to the USART_DR register. The RXNE flag can also be cleared by writing a zero to it. This clearing sequence is recommended only for multibuffer communication.

0: Data is not received

1: Received data is ready to be read.

수신 Data가 없거나 or 받는 중임

Data 수신 완료

```
uint8_t USART_getc(USART_TypeDef* USARTx) {
    // Wait until RXNE (RX not empty) bit is set by HW -->Read to read
    while ((USARTx->SR & USART_SR_RXNE) != USART_SR_RXNE);
    // Reading USART_DR automatically clears the RXNE flag
    return ((uint8_t)(USARTx->DR & 0xFF));
}
```

Transmit and receive data

- **Reading / Transmitting Data**

  2. Transmit or Receive the data **(USARTx_DR)**
  - RX and TX share the same data register

19.6.2 Data register (USART_DR)

Address offset: 0x04

Reset value: 0xXXXX XXXX

Bits 31:9 Reserved, must be kept at reset value

Bits 8:0 **DR[8:0]**: Data value

Contains the Received or Transmitted data character, depending on whether it is read from or written to.

The Data register performs a double function (read and write) since it is composed of two registers, one for transmission (TDR) and one for reception (RDR)

The TDR register provides the parallel interface between the internal bus and the output shift register (see Figure 1).

The RDR register provides the parallel interface between the input shift register and the internal bus.

When transmitting with the parity enabled (PCE bit set to 1 in the USART_CR1 register), the value written in the MSB (bit 7 or bit 8 depending on the data length) has no effect because it is replaced by the parity.

When receiving with the parity enabled, the value read in the MSB bit is the received parity bit.

550p

USART Receive Interrupt Setting

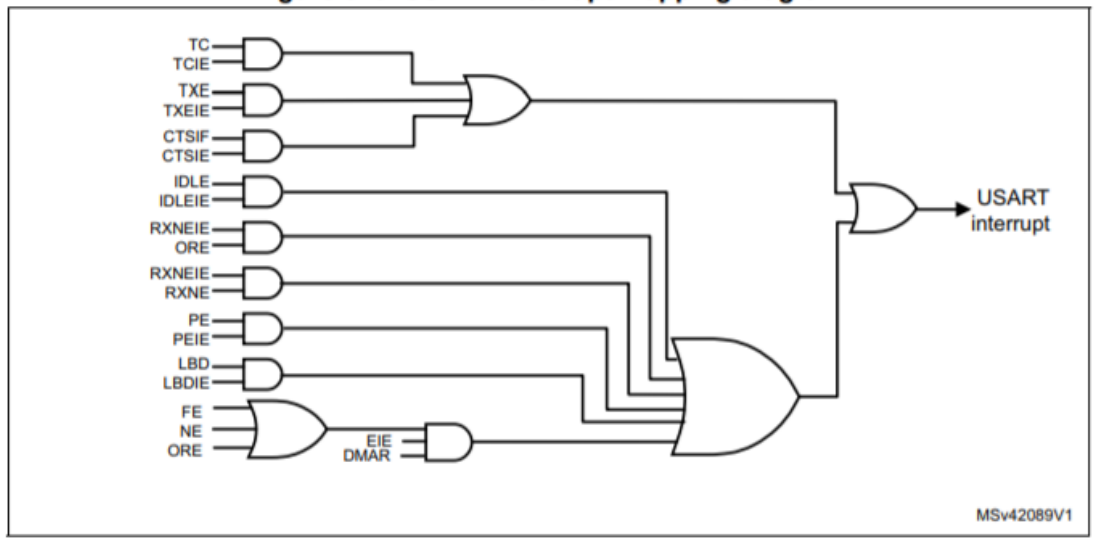
- **Interrupt USART**
  - For RX
    - RXNE, ORE
  - For TX
    - TXE, TC

30.4 USART interrupts

Table 147. USART interrupt requests

Interrupt event	Event flag	Enable control bit
Transmit Data Register Empty	TXE	TXEIE
CTS flag	CTS	CTSIE
Transmission Complete	TC	TCIE
Received Data Ready to be Read	RXNE	RXNEIE
Overrun Error Detected	ORE	
Idle Line Detected	IDLE	IDLEIE
Parity Error	PE	PEIE
Break Flag	LBD	LBDIE
Noise Flag, Overrun error and Framing Error in multibuffer communication	NF or ORE or FE	EIE

Figure 320. USART interrupt mapping diagram



USART Receive Interrupt Setting

- **Interrupt USART**

  (For Receive)

  1. Enable RXNE(Read data register not empty) interrupt. **(USARTx → CR1 : RXNEIE)**
  2. Set priority of and enable USARTx interrupt request with NVIC.

19.6.4 Control register 1 (USART_CR1)

Address offset: 0x0C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OVER8	Reserved	UE	M	WAKE	PCE	PS	PEIE	TXEIE	TCIE	RXNEIE	IDLEIE	TE	RE	RWU	SBK
rw	Res.	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 5 **RXNEIE**: RXNE interrupt enable

This bit is set and cleared by software.

0: Interrupt is inhibited

1: An USART interrupt is generated whenever ORE=1 or RXNE=1 in the USART_SR register

```
USARTx->CR1 |= USART_CR1_RXNEIE;

NVIC_SetPriority(USART2_IRQn, 1);    // Set Priority to 1
NVIC_EnableIRQ(USART2_IRQn);         // Enable interrupt of USART2 peripheral
```

```
void USART2_IRQHandler(void){
  // User code here---------------

}
```